

LPII - Cadastro com Componentes Adicionais

1 - Introdução

Neste tutorial, serão introduzidos: (a) novos componentes gráficos; (b) utilização de `ComboBox` com valores fixos; e (c) a representação de múltiplas referências entre objetos.

Atributos poderão ser melhor representados com a utilização dos seguintes componentes gráficos:

- `CheckBox` : para representar atributos booleanos;
- grupo de botões de rádio (`RadioButton`) : para representar enumerados com pouco elementos (máximo de 5);
- `ComboBox` com valores fixos : para representar enumerados com pouco elementos (acima de 5).

O componente `List` tem uma função semelhante ao `ComboBox`, suportando a inserção e remoção de elementos, mas mostrando vários elementos ao mesmo tempo; diferentemente do `ComboBox` que precisa ser aberto para mostrar seus elementos. O `List` será utilizado para mostrar os relacionamentos de um objeto com vários objetos de uma outra entidade.

As seções 2 e 3 deste tutorial, são utilizadas para caracterizar a atualização das janelas do Tutorial 1, para a representação mais adequada de atributos baseados em enumerados com poucos valores ou booleanos.

Na seção 4, será descrita a janela de cadastro da entidade `Filme` que tem um relacionamento de muitos para muitos (`n:n`) com a entidade `Ator`. Observe que: (a) em um filme atuam vários atores; e (b) um ator pode atuar em vários filmes.

Na seção 5, serão comentadas as diferenças que ocorrem em relacionamentos de um para muitos (`1:n`), utilizando como exemplo a relação entre as entidades `Montadora` e `Veículo`. Observe que: (a) uma montadora fabrica vários veículos; mas (b) um veículo é fabricado por uma única montadora.

2 – Atualizando a JanelaCadastrAmigos

A `JanelaCadastrAmigos` será atualizada com grupo de botões de rádio para representar os atributos `sexo` e `estado_civil`. A visualização da janela é mostrada a seguir.

A imagem mostra a interface de uma janela intitulada "Cadastrar Amigos". No topo, há uma barra de título com o ícone de uma pasta e o texto "Cadastrar Amigos". Abaixo, há um menu suspenso rotulado "Amigos Cadastrados" com o valor "Francisco Silveira [Chico Bento]". O formulário contém os seguintes campos e controles:

- Nome:** Campo de texto com o valor "Francisco Silveira".
- Apelido:** Campo de texto com o valor "Chico Bento".
- Sexo:** Grupo de botões de rádio com opções "feminino" (desselecionado) e "masculino" (selecionado).
- Estado Civil:** Grupo de botões de rádio com opções "solteiro" (desselecionado), "casado" (selecionado), "divorciado" (desselecionado) e "viúvo" (desselecionado).
- Cidade:** Campo de texto com o valor "Dourados".
- Email:** Campo de texto com o valor "francisco.silveira@gmail.com".
- Whatsapp:** Campo de texto com o valor "67-92222-2222".

Na base do formulário, há uma barra contendo cinco botões: "Inserir", "Consultar", "Alterar", "Remover" e "Limpar".

Inicialmente vamos comentar como configurar o componente `RadioButton` no NetBeans. A seguir, vamos comentar as alterações nos métodos: `obterAmigoInformado`, `consultarAmigo` e `limparCampos`.

2.1 - Configurando os Botões de Radio (RadioButton)

Cada elemento de um enumerado é representado por um componente `RadioButton`. Para os elementos do enumerado sejam agrupados no layout da janela, eles devem ser encapsulados por um componente `Panel`, da mesma forma que os componentes `Button` utilizados para representar os comandos de: inserção, consulta, etc.

Para que o grupo de botões de rádio tenha a função de ativar somente um elemento por vez, todos os botões deverão ser vinculados a um componente `ButtonGroup`. Na aba do Navegador, que mostrar a hierarquia dos componentes da página, o componente `ButtonGroup` fica vinculado ao diretório `Other Components`, caracterizando desta forma que esse componente não tem representação gráfica.

A configuração das propriedades específicas de cada `RadioButton` será exemplificada para o atributo `sexo`. Para o sexo feminino o primeiro `RadioButton` será configurado da seguinte forma: (a) `text`: feminino; (b) `buttonGroup`: `sexoButtonGroup` (criado previamente). Para o sexo masculino, a configuração é equivalente, acrescentando a propriedade `mnemonic`: `\u0001`. Quando o enumerado tem mais de 2 elementos, a configuração de `mnemonic` prossegue com `\u0002`, etc.

2.2 - Atualizando o método obterAmigoInformado

O método `obterAmigoInformado` é atualizado para a leitura dos enumerados associados aos atributos `sexo` e `estado_civil`.

```
private Amigo obterAmigoInformado() {
    String nome = nomeTextField.getText();
    if (nome.isEmpty()) return null;
    String apelido = apelidoTextField.getText();
    if (apelido.isEmpty()) apelido = null;
    String cidade = cidadeTextField.getText();
    if (cidade.isEmpty()) return null;
    String email = emailTextField.getText();
    if (email.isEmpty()) return null;
    Sexo sexo = null;
    if (sexoButtonGroup.getSelection() != null)
        sexo = Sexo.values()[sexoButtonGroup.getSelection().getMnemonic()];
    else return null;
    EstadoCivil estado_civil = null;
    if (estado_civilButtonGroup.getSelection() != null) {
        estado_civil = EstadoCivil.values()
            [estado_civilButtonGroup.getSelection().getMnemonic()];
    } else return null;
    String whatsapp = whatsappTextField.getText();
    if (whatsapp.isEmpty()) whatsapp = null;
    return new Amigo(nome, apelido, cidade, email, sexo, estado_civil, whatsapp);
}
```

O método `getSelection` da classe `ButtonGroup` retorna o objeto `RadioButton` selecionado. O método `getMnemonic` retorna o inteiro associado ao `RadioButton`. Antes de obter o mnemônico é necessário testar se algum `ButtonGroup` foi selecionado.

2.3 - Atualizando o método consultarAmigo

O método `obterAmigoInformado` é atualizado para a informar os valores dos enumerados associados aos atributos `sexo` e `estado_civil`.

```
private void consultarAmigo(java.awt.event.ActionEvent evt) {
    Amigo visão = (Amigo) amigos_cadastradosComboBox.getSelectedItem();
    Amigo amigo = null;
    String mensagem_erro = null;
    if (visão != null) {
        amigo = Amigo.buscarAmigo(visão.getNome());
        if (amigo == null) mensagem_erro = "Amigo não cadastrado";
    } else mensagem_erro = "Nenhum amigo selecionado";
    if (mensagem_erro == null) {
        nomeTextField.setText(amigo.getNome());
        String apelido = amigo.getApelido();
        if (apelido == null) apelido = "";
        apelidoTextField.setText(apelido);
        cidadeTextField.setText(amigo.getCidade());
        emailTextField.setText(amigo.getEmail());
        selecionarSexoRadioButton(amigo.getSexo().ordinal());
        selecionarEstadoCivilRadioButton(amigo.getEstadoCivil().ordinal());
        String whatsapp = amigo.getWhatsapp();
        if (whatsapp == null) whatsapp = "";
        whatsappTextField.setText(whatsapp);
    } else informarErro(mensagem_erro);
}
```

O método [obterAmigoInformado](#) utiliza métodos auxiliares, para selecionar o [RadioButton](#) correspondente ao elemento do enumerado associado aos atributos [sexo](#) e [estado_civil](#). A implementação do método auxiliar [selecionarEstadoCivilRadioButton](#) é ilustrada a seguir.

```
private void selecionarEstadoCivilRadioButton(int indice_estado_civil) {
    switch(indice_estado_civil) {
        case 0: solteiroRadioButton.setSelected(true);
            break;
        case 1: casadoRadioButton.setSelected(true);
            break;
        case 2: divorciadoRadioButton.setSelected(true);
            break;
        case 3: viuvoRadioButton.setSelected(true);
    }
}
```

A implementação do método auxiliar [selecionarSexoRadioButton](#) é equivalente.

2.4 - Atualizando o método limparCampos

O método [limparCampos](#) é atualizado para a remover as seleções dos valores dos enumerados associados aos atributos [sexo](#) e [estado_civil](#).

```
private void limparCampos(java.awt.event.ActionEvent evt) {
    nomeTextField.setText("");
    apelidoTextField.setText("");
    cidadeTextField.setText("");
    emailTextField.setText("");
    sexoButtonGroup.clearSelection();
    estado_civilButtonGroup.clearSelection();
    whatsappTextField.setText("");
}
```

3 – Atualizando a JanelaCadastroAtores

A [JanelaCadastroAtores](#) será atualizada com grupo de botões de rádio para representar os atributos [sexo](#) e [ganhador_oscar](#). A visualização da janela é mostrada a seguir.

A imagem mostra a interface gráfica da janela "Cadastrar Atores". No topo, há uma barra de título com o ícone de uma seta para cima e o texto "Cadastrar Atores". Abaixo, há uma seção "Atores Cadastrados" com um campo de texto contendo "Kate Winslet [1975]" e uma seta para baixo. Segue-se o campo "Nome" com o texto "Kate Winslet". Abaixo disso, o campo "Ano de Nascimento" contém "1975". O campo "Sexo" possui dois botões de rádio: "feminino" (selecionado) e "masculino". Abaixo, o campo "Ganhador de Oscar" possui um botão de checkbox marcado com um checkmark. Na base da janela, há uma barra com cinco botões: "Inserir", "Consultar", "Alterar", "Remover" e "Limpar".

A configuração do componente [CheckBox](#) é muito simples. São configuradas as seguintes propriedades específicas: (a) [text](#): Ganhador de Oscar; e (b) [horizontalTextPosition](#): [LEFT](#). As alterações nos métodos também são muito simples.

No método `obterAtorInformado` as alterações são as seguintes:

```
boolean ganhador_oscar = ganhador_oscarCheckBox.isSelected();
return new Ator (nome, ano_nascimento, sexo, ganhador_oscar);
```

No método `consultarAtor` utilizamos o comando:

```
ganhador_oscarCheckBox.setSelected(ator.isGanhadorOscar());
```

No método `limparCampos` utilizamos o comando:

```
ganhador_oscarCheckBox.setSelected(false);
```

4 – Relacionamento de Muitos para Muitos (n:n) entre Filme e Ator

O relacionamento entre as entidades `Filme` e `Ator` é de muitos para muitos (n:n), dado que: (a) em um filme podem atuar vários atores; e (b) em um ator pode atuar em vários filmes. Em um banco de dados relacional, para representar um relacionamento n:n é necessário criar uma entidade adicional, para interligar `Filme` e `Ator`, que vamos denominar como `Atuação`. Observe como fica o script sql para criar um banco de dados no Tutorial 2, omitindo criação das tabelas Amigos e Atores, cujo script é o mesmo do Tutorial 1. Na tabela Atuações, cada atuação tem uma coluna associada a uma chave estrangeira (FOREIGN KEY) referenciando (REFERENCES) a chave de um filme e de um ator.

```
DROP TABLE IF EXISTS Atuações;
DROP TABLE IF EXISTS Amigos;
DROP TABLE IF EXISTS Atores;
DROP TABLE IF EXISTS Filmes;

CREATE TABLE Filmes (
    Sequential INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    Título VARCHAR(60) NOT NULL,
    Gênero INT NOT NULL,
    Ano INT NOT NULL);

CREATE TABLE Atuações (
    Sequential INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    FilmeId INT NOT NULL,
    AtorId VARCHAR(50) NOT NULL,
    FOREIGN KEY (FilmeId) REFERENCES Filmes(Sequential),
    FOREIGN KEY (AtorId) REFERENCES Atores(Nome));
```

Neste caso, é necessário implementar o trio janela, controlador e entidade tanto para `Filme`, quanto para `Atuação`. Nas próximas subseções, são ilustradas a `JanelaCadastroFilmes` e a `JanelaCadastroAtuações`, com suas respectivas classes vinculadas.

4.1 - Construindo a JanelaCadastrFilmes e suas classes vinculadas

Os enumerados e atributos da entidade **Filme** são os seguintes:

```
public enum Gênero { ação, aventura, comédia, drama, faroeste, ficção, guerra, infantil,
    musical, romance, suspense, terror }

private int sequencial;
private String titulo;
private Gênero gênero;
private int ano;
```

Observe que não é definido nenhum dado como referência, dado que isto vai ocorrer somente na classe **Atuação**. Além do construtor com todos os atributos, é definido também um construtor para criar uma visão da classe **Filme** que será utilizado para povoar o **ComboBox** com os filmes cadastrados, que recebe como parâmetros a chave **sequencial** e o atributo **título**.

A ilustração da **JanelaCadastrFilmes** é mostrada a seguir. O **Button** com o título **Atuações**, é utilizado para visualizar a **JanelaCadastrAtuações** e selecionar os atores que atuam em um dado filme. Para a seleção do gênero do filme, é utilizado um **ComboBox**, dado que a quantidade de elementos do enumerado **Gênero**, da classe **Filme**, não pode ser adequadamente representada por botões de radio.

Nas próximas subseções serão comentados os tratadores de eventos cujos códigos requerem detalhes específicos de implementação. Os demais tratadores, **alterarFilme** e **removerFilme**, seguem o mesmo padrão das janelas comentadas anteriormente:

Na configuração do **ComboBox** utilizado para selecionar os valores fixos do enumerado **Gênero** da classe **Filme**, na propriedade **model** é informado, o seguinte código:

```
new DefaultComboBoxModel (Gênero.values())
```

A função estática **values** retorna os valores do enumerado que são associados ao **ComboBox**.

O [sequencial](#), utilizado com chave de cada filme, é criado automaticamente pelo banco de dados. Desta forma, o [TextField](#) utilizado para mostrar o [sequencial](#) é desabilitado para preenchimento pelo usuário; e seu valor só será atualizado por programa, após a criação do filme no banco de dados. Para desabilitar o [TextField](#) para edição é necessário desabilitar a sua propriedade: [editable](#).

Para informar os nomes dos principais atores que atuaram no filme é utilizado um componente [List](#), que diferentemente do [ComboBox](#) mostra mais de um elemento ao mesmo tempo. Esse componente sempre é inserido em um componente container [ScrollPane](#), para que caso o número de linhas ou colunas mostrado no [List](#) exceda o tamanho configurado, apareça respectivamente um scrollbar horizontal ou vertical, dependendo de qual dimensão foi excedida.

A configuração do [List](#) é diferente da configuração do [ComboBox](#). A sua propriedade [visibleRowCount](#) foi configurada com 4, indicando que o [List](#) mostra até 4 linhas. Caso ultrapasse, aparecerá o scrollbar vertical. A sua propriedade [selectionMode](#) é configurada com [SINGLE](#), indicando que somente uma linha do [List](#) será selecionada por vez. A sua propriedade [model](#) é configurada com o seguinte código:

```
new DefaultListModel()
```

Para o [List](#), as visões são associadas diretamente no seu modelo e não no componente gráfico. Por esse motivo o construtor da [JanelaCadastroFilmes](#) armazena o modelo que será utilizado posteriormente.

```
ControladorCadastroFilmes controlador;
Filme[] filmes_cadastrados;
DefaultListModel modelo_atuações_filme;

public JanelaCadastroFilmes(ControladorCadastroFilmes controlador) {
    this.controlador = controlador;
    filmes_cadastrados = Filme.getVisões();
    initComponents();
    modelo_atuações_filme = (DefaultListModel)atuações_filmeList.getModel();
    limparCampos(null);
}
```

O método [getVisões](#), da classe [Filme](#), é semelhante aos das classes [Amigo](#) e [Ator](#) definidos anteriormente. Ao [Button](#) com texto [Atuações](#), é vinculado o tratador de eventos [cadastrarAtuações](#).

4.1.1 - O Tratamento de Eventos para Inserir um Filme

A seguir, é ilustrada a implementação do tratador de eventos [inserirFilme](#).

```
private void inserirFilme(java.awt.event.ActionEvent evt) {
    Filme filme = obtémFilmeInformado();
    String mensagem_erro = null;
    if (filme != null) mensagem_erro = controlador.inserirFilme(filme);
    else mensagem_erro = "Algum atributo do filme não foi informado";
    if (mensagem_erro == null) {
        int sequencial = Filme.últimoSequencial();
        filme.setSequencial(sequencial);
        Filme visão = filme.getVisão();
        filmes_cadastradosComboBox.addItem(visão);
        filmes_cadastradosComboBox.setSelectedItem(visão);
        sequencialTextField.setText (" " + sequencial);
    } else informarErro (mensagem_erro);
}
```

Após a criação na base de dados, deve ser lido o último [sequencial](#) criado, para atualizar o atributo [sequencial](#) do objeto filme e mostrá-lo no [sequencialTextField](#).

O método [obterFilmeInformado](#), é implementado da seguinte forma:

```
private Filme obtémFilmeInformado() {
    String sequencial_str = sequencialTextField.getText();
    int sequencial = 0;
    if (!sequencial_str.isEmpty()) sequencial = Integer.parseInt(sequencial_str);
    String título = títuloTextField.getText();
    if (título.isEmpty()) return null;
    Gênero gênero = null;
    if (gêneroComboBox.getSelectedItem() != null)
        gênero = (Gênero)gêneroComboBox.getSelectedItem();
    else return null;
    String ano_str = anoFormattedTextField.getText();
    int ano = -1;
    if (!ano_str.isEmpty()) ano = Integer.parseInt(ano_str);
    else return null;
    return new Filme(sequencial, título, gênero, ano);
}
```

O método [obterFilmeInformado](#) é utilizado pelos tratadores de evento: [inserirFilme](#) e [alterarFilme](#). Antes de inserir um filme, o sequencial ainda não foi criado pelo banco. Neste caso, o filme é criado com o [sequencial](#) com valor 0, que na realidade não existe, porque os sequenciais são criados pelo banco de dados a partir do valor 1.

Dado que o [sequencialComboBox](#) está desabilitado para preenchimento, para alterar um filme é necessário consultá-lo, e converter o string lido em um número inteiro, para criar o objeto filme utilizando com o [sequencial](#) obtido.

Para ler o valor do [gênero](#) do filme, informado no [gêneroComboBox](#), é necessário primeiro testar se algum valor foi selecionado no [ComboBox](#).

O método estático [últimoSequencial](#), da classe [Filme](#), é implementado levando em conta o fato que o maior [sequencial](#) utilizado na tabela [Filmes](#) é o último [sequencial](#) criado.

```
public static int últimoSequencial () {
    String sql = "SELECT MAX(Sequencial) FROM Filmes";
    ResultSet lista_resultados = null;
    int sequencial = 0;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            sequencial = lista_resultados.getInt (1);
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return sequencial;
}
```

A implementação do método [inserirFilme](#), da classe [ControladorCadastroFilmes](#), é a seguinte:

```
public String inserirFilme (Filme filme) {
    if (!Filme.existeFilmeMesmosAtributos (filme)) return Filme.inserirFilme (filme);
    else return "Já existe um filme com os mesmos atributos";
}
```


Dado que o [sequencial](#) é criado automaticamente, se o usuário informasse os mesmos dados de um filme já criado, o filme seria criado novamente no banco com um outro [sequencial](#). Para evitar que isso aconteça, o controlador verifica se já existe um filme com os mesmos atributos, obviamente desconsiderando o [sequencial](#), que neste momento está assinalado provisoriamente como 0 no objeto filme recebido. A implementação do método estático [existeFilmeMesmosAtributos](#), da classe [Filme](#), é baseada na contagem do quantos sequenciais distintos foram criados para um filme com os mesmos atributos. Se for 0, é porque nenhum filme ainda foi criado com os mesmos atributos. Se for 1, é porque já existe um filme. Observe que nunca será superior a 1, porque o controlador não permite a criação de mais de um filme com os mesmos atributos.

```
public static boolean existeFilmeMesmosAtributos (Filme filme) {
    String sql = "SELECT COUNT(Sequencial) FROM Filmes"
        + " WHERE Título = ? AND Gênero = ? AND Ano = ?";
    ResultSet lista_resultados = null;
    int n_filmes_mesmos_atributos = 0;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setString(1, filme.getTítulo());
        comando.setInt(2, filme.getGênero().ordinal());
        comando.setInt(3, filme.getAno());
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            n_filmes_mesmos_atributos = lista_resultados.getInt(1);
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    if (n_filmes_mesmos_atributos > 0) return true;
    else return false;
}
```

A implementação dos métodos estáticos [buscarFilme](#) e [inserirFilme](#), da classe [Filme](#), são equivalentes aos métodos definidos anteriormente.

4.1.2 - O Tratamento de Eventos para Consultar um Filme

A seguir, é ilustrada a implementação do tratador de eventos [consultarFilme](#).

```
private void consultarFilme(java.awt.event.ActionEvent evt) {
    Filme visão = (Filme) filmes_cadastradosComboBox.getSelectedItem();
    Filme filme = null;
    String mensagem_erro = null;
    int sequencial = -1;
    if (visão != null) {
        sequencial = visão.getSequencial();
        filme = Filme.buscarFilme(sequencial);
        if (filme == null) mensagem_erro = "Filme não cadastrado";
    } else mensagem_erro = "Nenhum filme selecionado";
    if (mensagem_erro == null) {
        sequencialTextField.setText(sequencial + "");
        títuloTextField.setText (filme.getTítulo());
        gêneroComboBox.setSelectedItem(filme.getGênero());
        anoFormattedTextField.setText (filme.getAno() + "");
        atualizarListaAtuaçõesFilme(sequencial);
    } else informarErro (mensagem_erro);
}
```

O método [atualizarListaAtuaçõesFilme](#) é implementado da seguinte forma:

```
public void atualizarListaAtuaçõesFilme (int sequencial) {
    modelo_atuações_filme.clear();
    Atuação[] atuações_filme = Atuação.buscarAtuaçõesFilme(sequencial);
    for (Atuação atuação : atuações_filme) modelo_atuações_filme.addElement(atuação);
}
```

O método [buscarAtuaçõesFilme](#), da classe [Atuação](#), é ilustrado a seguir:

```
public static Atuação[] buscarAtuaçõesFilme (int sequencial_filme) {
    String sql = "SELECT Sequencial, AtorId FROM Atuações WHERE FilmeId = ?";
    ResultSet lista_resultados = null;
    ArrayList<Atuação> visões = new ArrayList();
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial_filme);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) {
            visões.add(new Atuação (lista_resultados.getInt("Sequencial"),
                Filme.buscarFilme(sequencial_filme).getVisão(),
                Ator.buscarAtor(lista_resultados.getString("AtorId")).getVisão()));
        }
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {exceção_sql.printStackTrace ();}
    return visões.toArray(new Atuação[visões.size()]);
}
```

4.1.3 - O Tratamento de Eventos para Limpar Campos

A seguir, é ilustrada a implementação do tratador de eventos [limparCampos](#).

```
private void limparCampos(java.awt.event.ActionEvent evt) {
    sequencialTextField.setText("");
    títuloTextField.setText("");
    gêneroComboBox.setSelectedIndex(-1);
    anoFormattedTextField.setText("");
    modelo_atuações_filme.clear();
}
```

4.1.4 - O Tratamento de Eventos para Cadastrar Atuações

A seguir, é ilustrada a implementação do tratador de eventos [cadastrarAtuações](#).

```
private void cadastrarAtuações(java.awt.event.ActionEvent evt) {
    String sequencial_str = sequencialTextField.getText();
    int sequencial = 0;
    if (!sequencial_str.isEmpty()) sequencial = Integer.parseInt(sequencial_str);
    if (sequencial > 0) new ControladorCadastroAtuaçõesFilme(this, sequencial);
    else informarErro ("Nenhum filme selecionado");
}
```

O construtor do ControladorCadastroAtuaçõesFilme é ilustrado na seção 4.2.

4.2 - Construindo a JanelaCadastroAtuações e suas classes vinculadas

Os atributos da entidade [Atuação](#) são os seguintes:

```
private int sequencial;  
private Filme filme;  
private Ator ator;
```

Como informado anteriormente, na classe [Filme](#) não é definida nenhuma referência para a classe [Ator](#). Em um relacionamento n:n, as referências ficam somente na nova classe criada para associar os objetos das entidades relacionadas; neste caso, a classe [Atuação](#).

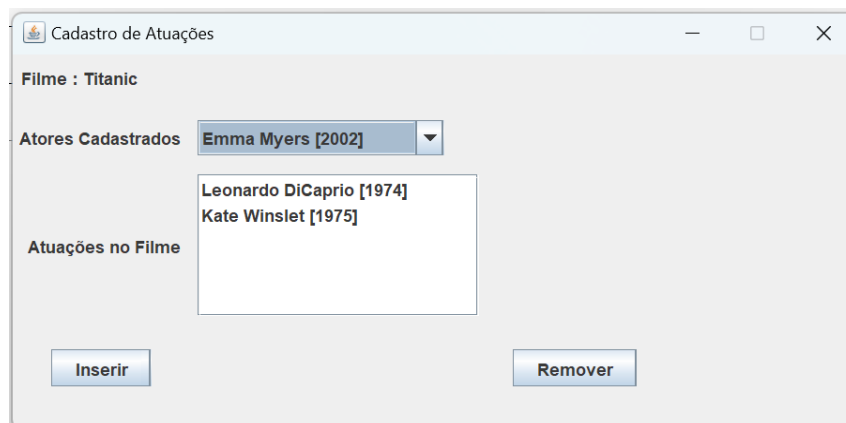
Conforme ilustrado na seção 4.1.4, o tratador de eventos [cadastrarAtuações](#) da [JanelaCadastroFilmes](#) cria um objeto do [ControladorCadastroAtuações](#).

A implementação do construtor desse controlador é ilustrada a seguir.

```
public ControladorCadastroAtuaçõesFilme(JanelaCadastroFilmes janela_cadastro_filmes,  
    int sequencial_filme)  
{  
    new JanelaCadastroAtuaçõesFilme(this, janela_cadastro_filmes, sequencial_filme)  
        .setVisible(true);  
}
```

O construtor do controlador recebe como argumentos: (a) a referência ao objeto da [JanelaCadastroFilmes](#) que o executou; e (b) o [sequencial](#) que identifica o filme para o qual serão criadas as atuações.

A ilustração da [JanelaCadastroAtuações](#) é mostrada a seguir.



Nas subseções, são descritos os tratadores de evento associados a essa janela. Os dados da janela, seu construtor e seus métodos auxiliares são mostrados a seguir.

```

ControladorCadastroAtuaçõesFilme controlador;
JanelaCadastroFilmes janela_mãe;
int sequencial_filme;
Ator[] atores_cadastrados;
DefaultListModel modelo_lista_atuações;

public JanelaCadastroAtuaçõesFilme(ControladorCadastroAtuaçõesFilme controlador,
    JanelaCadastroFilmes janela_mãe, int sequencial_filme) {
    this.controlador = controlador;
    this.janela_mãe = janela_mãe;
    this.sequencial_filme = sequencial_filme;
    atores_cadastrados = Ator.getVisões();
    initComponents();
    atualizarTítuloFilme();
    atualizarListaAtuaçõesFilme();
}

private void atualizarTítuloFilme () {
    Filme filme = Filme.buscarFilme(sequencial_filme);
    filmeLabel.setText("Filme : " + filme.getTítulo());
}

private void atualizarListaAtuaçõesFilme () {
    modelo_lista_atuações = (DefaultListModel)atuações_filmeList.getModel();
    Atuação[] atuações_filme = Atuação.buscarAtuaçõesFilme(sequencial_filme);
    for (Atuação atuação : atuações_filme) modelo_lista_atuações.addElement(atuação);
}

```

O construtor da janela recebe como argumentos:

- o objeto chamador do [ControladorCadastroAtuações](#), para que possa chamá-lo nos tratadores de evento: [inserirAtuação](#) e [removerAtuação](#);
- objeto da [JanelaCadastroFilmes](#), para que possa chamar seu método [atualizarListaAtuaçõesFilme](#) no tratador de evento: [atualizarJanelaCadastroFilmes](#);
- e o [sequencial](#) do filme para o qual serão cadastradas as atuações.

O método auxiliar [atualizarTítuloFilme](#) atualiza o [filmeLabel](#) com o título do filme, para o qual serão cadastradas as atuações. O método auxiliar [atualizarListaAtuaçõesFilme](#), atualiza o modelo do [atuaçõesList](#) com as atuações cadastradas previamente.

4.2.1 - O Tratamento de Eventos para Inserir uma Atuação

A implementação do tratador de eventos [inserirAtuação](#) é ilustrada a seguir.

```

private void inserirAtuação(java.awt.event.ActionEvent evt) {
    Ator visão_ator = (Ator) atores_cadastradosComboBox.getSelectedItem();
    String mensagem_erro = null;
    Atuação atuação = null;
    if (visão_ator != null) {
        Filme visão_filme = Filme.buscarFilme(sequencial_filme);
        atuação = new Atuação (0, visão_filme, visão_ator);
        mensagem_erro = controlador.inserirAtuação(atuação);
    } else mensagem_erro = "Nenhum ator foi selecionado";
    if (mensagem_erro == null) {
        int sequencial = Atuação.últimoSequencial();
        atuação.setSequencial(sequencial);
        modelo_lista_atuações.addElement(atuação);
    } else informarErro (mensagem_erro);
}

```

A implementação do método `inserirAtuação`, do `ControladorCadastroAtuações`, é ilustrada a seguir.

```
public String inserirAtuação (Atuação atuação) {
    boolean existe_atuação = Atuação.existeAtuação
        (atuação.getFilme().getSequencial(), atuação.getAtor().getNome());
    if (!existe_atuação) return Atuação.inserirAtuação (atuação);
    else return "Sequencial de atuação já cadastrado";
}
```

A implementação do método `existeAtuação`, da classe `Atuação`, que recebe como argumentos o `sequencial` do filme e o `nome` do ator referenciados pela atuação, é ilustrado a seguir.

```
public static boolean existeAtuação (int chave_filme, String chave_ator) {
    String sql = "SELECT Sequencial FROM Atuações WHERE FilmeId = ? AND AtorId = ?";
    ResultSet lista_resultados = null;
    boolean existe = false;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, chave_filme);
        comando.setString(2, chave_ator);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) existe = true;
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) { exceção_sql.printStackTrace (); }
    return existe;
}
```

A implementação dos métodos `últimoSequencial` e `inserirAtuação`, da classe `Atuação`, é equivalente aos métodos de mesmo nome definido na classe `Filme`.

4.2.2 - O Tratamento de Eventos para Remover uma Atuação

A implementação do tratador de eventos `removerAtuação` é ilustrada a seguir.

```
private void removerAtuação(java.awt.event.ActionEvent evt) {
    Atuação atuação = (Atuação) atuações_filmeList.getSelectedValue();
    String mensagem_erro = null;
    if (atuação != null) mensagem_erro = controlador.removerAtuação(atuação.getSequencial());
    else mensagem_erro = "Nenhum ator foi selecionado";
    if (mensagem_erro == null) modelo_lista_atuações.removeElement(atuação);
    else informarErro (mensagem_erro);
}
```

A implementação do método `removerAtuação`, do `ControladorCadastroAtuações`, é ilustrada a seguir.

```
public String removerAtuação (int sequencial) {
    boolean existe_atuação = Atuação.existeAtuação (sequencial);
    if (existe_atuação) return Atuação.removerAtuação (sequencial);
    else return "Não existe atuação com este sequencial";
}
```

A implementação do método `existeAtuação`, da classe `Atuação`, que recebe como argumento o `sequencial` da atuação, é ilustrado a seguir.

```
public static boolean existeAtuação (int sequencial) {
    String sql = "SELECT COUNT(Sequencial) FROM Atuações WHERE Sequencial = ?";
    ResultSet lista_resultados = null;
    boolean existe = false;
    try {
        PreparedStatement comando = BD.conexão.prepareStatement(sql);
        comando.setInt(1, sequencial);
        lista_resultados = comando.executeQuery();
        while (lista_resultados.next()) existe = true;
        lista_resultados.close();
        comando.close();
    } catch (SQLException exceção_sql) {
        exceção_sql.printStackTrace ();
    }
    return existe;
}
```

A implementação do método `removerAtuação`, da classe `Atuação`, é equivalente ao método de mesmo nome definido na classe `Filme`.

4.2.3 - O Tratamento de Eventos para Atualizar a JanelaCadastroFilmes

Ao fechar a `JanelaCadastroAtuações`, as atuações cadastradas para um dado filme devem ser exibidas no `atuaçõesList` da classe `JanelaCadastroFilmes`. Para que isso ocorra, deve ser associado ao evento `windowClosed`, da `JanelaCadastroAtuações`, o tratador de eventos `atualizarJanelaCadastroFilmes` cuja implementação é ilustrada a seguir.

```
private void atualizarJanelaCadastroFilmes(java.awt.event.WindowEvent evt) {
    janela_mãe.atualizarListaAtuaçõesFilme(sequencial_filme); }
```

5 - Ilustrando as diferenças em um Relacionamento de Um para Muitos (1:n)

Existem situações, nas quais o relacionamento é de um para muitos (1:n) como, por exemplo, o relacionamento entre as entidades `Montadora` e `Veículo`: (a) uma montadora pode produzir vários veículos; e (b) um veículo é produzido por uma única montadora.

No relacionamento n:n entre `Filme` e `Ator`, foi necessário criar a entidade `Atuação` para relacionar um filme com um ator, dado que na tabela `Filmes` um filme não pode referenciar as chaves estrangeiras de vários atores, e vice-versa na tabela `Atores` com as chaves estrangeiras de vários filmes.

No entanto, no relacionamento 1:n entre **Montadora** e **Veículo**, um veículo da tabela **Veículos** pode referenciar a chave estrangeira da montadora que o fabricou. Portanto, não é necessário criar uma entidade adicional para caracterizar um relacionamento 1:n. Essa simplificação tem as seguintes consequências:

- na criação de um veículo, na **JanelaCadastroVeículo**, deve ser escolhida a montadora que o fabrica, utilizando o componente **montadoraComboBox** exibindo as visões das montadoras cadastradas;
- obviamente, o cadastro da montadora, que fabrica o veículo, deve ter sido realizado previamente na **JanelaCadastroMontadoras**;
- na consulta de uma montadora na **JanelaCadastroMontadoras**, o componente **veiculosList** (encapsulado pelo componente **veiculosPanel**) deve exibir as visões dos veículos que foram previamente associados à montadora consultada na **JanelaCadastroVeículos**;
- obviamente, quando uma montadora é criada, na **JanelaCadastroMontadoras**, o componente **veiculosList** estará vazio, só passando a exibir veículos após eles terem sido criados na **JanelaCadastroVeículos** associados a uma determinada montadora, a partir do método **consultarMontadora**, que por sua vez executa o método privado **atualizarListaVeiculosMontadora**, ilustrado a seguir.

```
public void atualizarListaVeiculosMontadora (int sequencial_montadora) {
    modelo_veiculos_montadora.clear();
    Veiculo[] veiculos_montadora = Veiculo.buscarVeiculosMontadora (int sequencial_montadora);
    for (Veiculo visão_veiculo : veiculos_montadora)
        modelo_veiculos_montadora.addElement(visão_veiculo);
}
```

O script SQL de criação das tabelas **Montadoras** e **Veículos** é ilustrado a seguir.

```
DROP TABLE IF EXISTS Veículos;
DROP TABLE IF EXISTS Montadoras;

CREATE TABLE Montadoras (
    Sequencial INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    Nome VARCHAR(60) NOT NULL,
    PaísOrigem VARCHAR(30) NOT NULL,
    SedeMaiorFábricaBrasil VARCHAR(30) NOT NULL);

CREATE TABLE Veiculos (
    MarcaModelo VARCHAR(15) NOT NULL PRIMARY KEY,
    Alimentação INT,
    Potência INT,
    Importado BOOLEAN,
    MontadoraId INT NOT NULL,
    FOREIGN KEY (MontadoraId) REFERENCES Montadoras(Sequencial));
```

A seguir são ilustrados na classe **Veículo**: (a) o método estático **buscarVeículosMontadora**; e (2) os atributos, os construtores e o método **toString**.

```
class Veículo {

    public static Veículo[] buscarVeículosMontadora (int sequencial_montadora) {
        String sql = null;
        ResultSet lista_resultados = null;
        ArrayList<Veículo> visões = new ArrayList();
        sql = "SELECT MarcaModelo, Alimentação FROM Veículos WHERE MontadoraId = ?";
        try {
            PreparedStatement comando = BD.conexão.prepareStatement(sql);
            comando.setInt(1, sequencial_montadora);
            lista_resultados = comando.executeQuery();
            while (lista_resultados.next()) {
                visões.add(new Veículo (lista_resultados.getString("MarcaModelo"),
                    Alimentação.values()[lista_resultados.getInt("Alimentação")] ));
            }
            lista_resultados.close();
            comando.close();
        } catch (SQLException exceção_sql) { exceção_sql.printStackTrace (); }
        return visões.toArray(new Veículo[visões.size()]);
    }

    public enum Alimentação { flex, diesel, elétrico };

    private String marca_modelo;
    private Alimentação alimentação;
    private int potência;
    private boolean importado;

    public Veículo (marca_modelo, alimentação, potência, importado) {
        this.marca_modelo = marca_modelo;
        this.alimentação = alimentação;
        this.potência = potência;
        this.importado = importado;
    }

    public Veículo (marca_modelo, alimentação) {
        this.marca_modelo = marca_modelo;
        this.alimentação = alimentação;
    }

    public String toString() {
        return marca_modelo + " [" + alimentação + "];"
    }

}
```